AUTOFORGE: A CHECKPOINTED, HUMAN–IN–THE–LOOP MULTI-AGENT SDLC PIPELINE

INDIVIDUAL THESIS

Dissanayake T.C.

IT22157232

Individual component: Code Generation with Sandboxed Verification

Group ID: R26-SE-033

Report submitted in partial fulfilment of the requirements for the

Bachelor of Science (Honours) in Information Technology

Specialising in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Dissanayake T.C

Student ID: IT22157232

Signature: _____ Date: ____05/10/2026_____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Nuwan Kodagoda		
Co-supervisor: Eishan Weerasinghe		10/10/2026

ABSTRACT

Coding agents can now write working code, but they usually report their own success, and a human reviewer has to take that report on trust. This thesis covers the Coder Agent of AutoForge, a seven-agent, human-gated pipeline for e-commerce web development. The agent turns an approved architecture plan and page designs into a code plan, which a validator checks against every endpoint, entity and functional requirement before any code is written. It then works on an isolated git branch of a Next.js and MongoDB project, using a tool-using coding loop when the model supports tool calls and a per-file batch path when it does not. Each attempt is committed and verified in a Docker sandbox: the project must install, build, type-check and boot, and deterministic checks confirm that planned routes exist, required database fields can be filled from a form, new pages are linked, and no connection string is hardcoded. Up to three attempts are made, each fed the previous failure. The merge report and diff shown to the reviewer are built from git, not from the model's account, and the branch is merged only on approval. I evaluated the agent with 410 automated tests, the team's three-feature case study and nine stored code versions from three features. Verification failures were real and specific, such as a missing dependency, a wrong import and form fields a schema required. Two failures came from Docker not running and were reported as failures rather than crashes. The stored runs also show what verification cannot see: a passing feature took image URLs instead of uploads, stored prices as text, and was later changed by hand before merging. A sandboxed build proves that code runs, not that it meets the specification, so human review of the real diff remains essential.

Keywords: code generation, coding agents, sandboxed verification, human-in-the-loop, multi-agent systems, git

ACKNOWLEDGEMENTS

I would like to thank my supervisor, Nuwan Kodagoda, and my co-supervisor, Eishan Weerasinghe, for their guidance throughout this project, and in particular for asking how we could prove that generated code actually works rather than relying on the agent's own account.

I am grateful to my teammates, Santhuka D.N.M.D., Sansala T.G.B.D. and Oberathna R.D.T.D. Their specifications, plans and page designs were what my agent built from, and their Security and QA agents were the first to test what it produced. Several checks described in this thesis exist because one of them found a problem in generated code.

I also thank the academic staff of the Department of Software Engineering at the Sri Lanka Institute of Information Technology, and the panel members at the progress presentations, for their feedback. Finally, I thank my family and friends for their support during this project

TABLE OF CONTENTS

DECLARATION AND SUPERVISOR CERTIFICATION i

ABSTRACT ii

ACKNOWLEDGEMENTS iii

LIST OF FIGURES viii

LIST OF TABLES ix

LIST OF ABBREVIATIONS x

1. INTRODUCTION 1

1.1 Overall project context 1

1.2 Individual component context 2

1.3 Problem statement 2

1.4 Research gap 3

1.5 Aim and objectives 3

1.6 Research questions or hypotheses 4

1.7 Scope limitations and assumptions 4

1.8 Individual contribution 4

1.9 Thesis structure 5

2. LITERATURE REVIEW AND RELATED WORK 6

2.1 Review method and scope 6

2.2 Domain and theoretical foundations 6

2.2.1 Tool-using coding agents 6

2.2.2 Verification by execution 6

2.2.3 Version control as a safety mechanism 6

2.2.4 Human oversight of agent changes 7

2.2.5 Design science 7

2.3 Related research 7
2.4 Existing systems products and techniques 8
2.5 Gap synthesis and conceptual framework 9
3. RESEARCH METHODOLOGY 10
3.1 Research design 10
3.2 Data participants or study materials 10
3.3 Experimental or study procedure 11
3.4 Measures and evaluation metrics 11
3.5 Data analysis 12
3.6 Ethics privacy security and responsible AI 12
3.7 Reproducibility and research artefacts 12
4. PRODUCT AND SOLUTION DESIGN 14
4.1 Requirements 14
4.2 Component architecture within the complete solution 15
4.3 Design alternatives and decisions 16
4.4 Detailed design 17
4.4.1 Planning and plan validation 17
4.4.2 Workspace and branches 18
4.4.3 Coding loop and batch path 18
4.4.4 Attempts, gap checks and verification 20
4.4.5 Reporting, approval and merge 21
4.4.6 Revisions, security fixes and the live preview 22
4.5 Standards and professional practice 22
5. IMPLEMENTATION INTEGRATION AND TESTING 24
5.1 Development environment and technology stack 24
5.2 Component implementation 24

5.2.1 Agent orchestration 25
5.2.2 Tools and sandbox 25
5.2.3 Verifier and checkers 26
5.2.4 Workspace and diff builder 26
5.3 Integration with the other components 27
5.4 Verification and software testing 27
5.5 Deployment and operation 28
6. RESULTS AND VALIDATION 29
6.1 Research results 29
6.2 Product and technical results 29
6.2.1 Plan validation and alignment with the design (RQ1) 29
6.2.2 What sandboxed verification found (RQ2) 30
6.2.3 Retrying with feedback (RQ3) 32
6.2.4 Human review of the real diff (RQ4) 32
6.3 User or stakeholder validation 33
6.4 Integration results 34
6.5 Summary of findings 34
7. DISCUSSION 35
7.1 Interpretation of findings 35
7.2 Comparison with literature baselines and existing solutions 35
7.3 Research contribution 36
7.4 Product contribution 37
7.5 Limitations and threats to validity 39
7.5.1 Threats to Validity 39
7.5.2 Limitations 39
7.6 Implications 40

8. COMMERCIALISATION AND SUSTAINABILITY 42
8.1 Value of the Component 42
8.2 Target Users and Market Position 43
8.3 Business Model and Feasibility 44
8.4 Sustainability and Maintenance 45
8.5 Contribution to the Sustainable Development Goals 46
9. CONCLUSION AND FUTURE DIRECTIONS 47
9.1 Conclusion 47
9.2 Achievement of objectives 47
9.3 Individual research and product contributions 47
9.4 Future directions 48
REFERENCES 49
GLOSSARY 52
APPENDICES 53
Appendix A Research instruments or extended results 53
A.1 Test catalogue 53
A.2 Verification summary of the retail store feature, version 1 53
Appendix B Academic integrity and AI use disclosure 54

LIST OF FIGURES

Figure 1.1: The AutoForge pipeline, showing the seven agents, their outputs, human approval and feedback loops 1
Figure 2.1: Conceptual framework for the coding component 9
Figure 4.1:The component's place in the complete AutoForge architecture..... 16
Figure 4.2:Coder Agent flow from plan to merge..... 19
Figure 4.3: Hard and informational verification steps 21

LIST OF TABLES

Table 2.1: Coding-stage properties of related systems 8
Table 3.1: Coder Agent runs analysed in this thesis 10
Table 3.2: Measures used for each research question 11
Table 3.3: Configuration parameters of the component 12
Table 4.1: Functional requirements of the component 14
Table 4.2: Non-functional requirements of the component 14
Table 4.3: Main design decisions 16
Table 4.4: Tools available to the coding loop 19
Table 5.1: Technology stack used by the component 24
Table 5.2: Main modules of the component 24
Table 5.3: Automated tests for the component 27
Table 6.1: Key behaviours confirmed by automated tests 29
Table 6.2: Verification outcome of the stored code versions 30
Table 6.3: Verified retail store code compared with the approved artefacts 31
Table 6.4: Pipeline with human gates on and off 33
Table 6.5: Execution performance of the full pipeline 34
Table 6.6: Summary of findings by research question 34
Table 8.1: Target users of the component 42
Table 8.2: Alternatives for producing feature code 43
Table 8.3: Revenue elements that depend on this component 44
Table 8.4: Work needed before a commercial release 45
Table 9.1: Achievement of the objectives 47
Table A.1:Test files for the component 54
Table B.1:Use of generative AI tools 55

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AC	Acceptance Criterion
AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update and Delete
CWE	Common Weakness Enumeration
FR	Functional Requirement
JSON	JavaScript Object Notation
LLM	Large Language Model
NFR	Non-Functional Requirement
npm	Node Package Manager
QA	Quality Assurance
REST	Representational State Transfer
RQ	Research Question
SDG	Sustainable Development Goal
SDLC	Software Development Life Cycle
SRS	Software Requirements Specification
UI	User Interface
URI	Uniform Resource Identifier
UX	User Experience

1. INTRODUCTION

1.1 Overall project context

AutoForge is our team's final-year research project: a multi-agent system that builds features of an e-commerce web application with large language models (LLMs). It works in stages, the way a software team would. A Requirement Agent writes the Software Requirements Specification (SRS), a Domain Agent enriches it, an Architecture Agent designs the solution, a UI/UX Agent designs the screens, a Coder Agent implements the change, and Security and Quality Assurance (QA) agents check the result. Each agent produces a versioned artefact, and a person approves, rejects or asks for changes before the next agent uses it.

Our research paper starts from a simple point. An LLM can write a working function on request, but a feature is more than a function, and tools that go straight from a prompt to code leave nothing to check when the result is wrong. Multi-agent frameworks such as ChatDev [1] and MetaGPT [2] split the work into roles, but review still happens through agent conversation or after the run. Figure 1.1 shows the AutoForge pipeline from the paper.

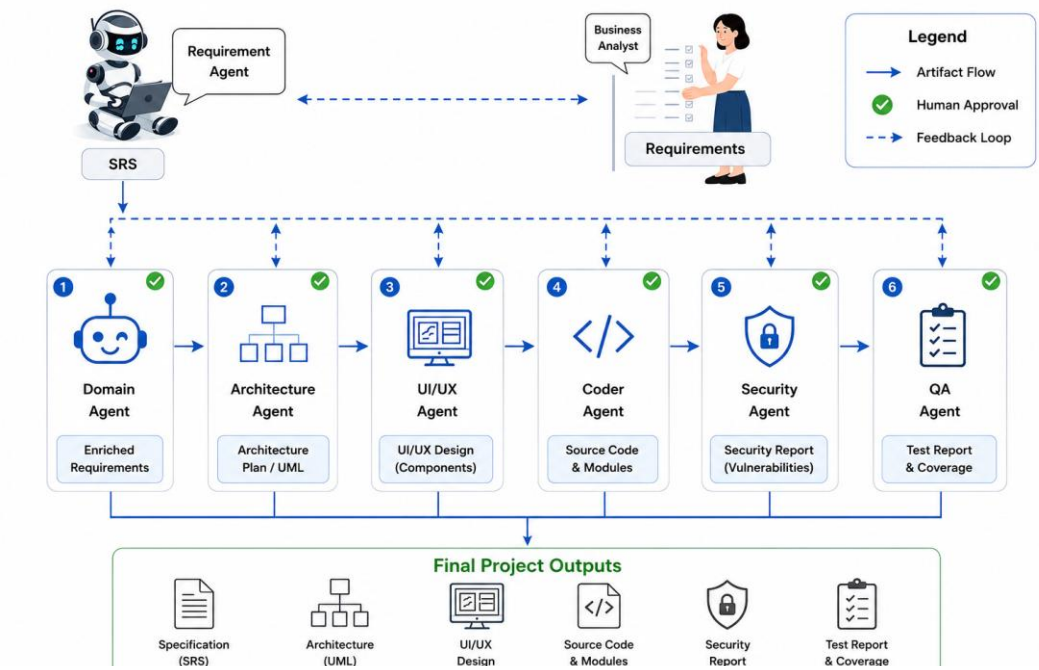


Figure 1.1: The AutoForge pipeline, showing the seven agents, their outputs, human approval and feedback loops

The project was divided into four components, one per team member. My component is the Coder Agent: the stage that turns approved designs into real code in a real project, and proves that the code builds and runs before a person decides whether to keep it. The other components cover requirements and orchestration, architecture and UI/UX design, and security and quality assurance. The team's common report describes the integrated system; this thesis describes my component in detail.

1.2 Individual component context

The coding stage is where mistakes become expensive. Up to this point every artefact is a document or a picture that a reviewer can read in minutes. From this point on, the output is a set of files in a repository, and a reviewer who approves it is approving changes to a running application. In e-commerce, that application stores customer accounts, product data and orders, so a careless change has real consequences.

This gives the Coder Agent three jobs. It must implement what the approved plan and page designs describe, not something it improvises. It must show that the result actually works, by building and running it, rather than by saying so. And it must make the change easy to review and easy to undo, so that approving or rejecting it is a clean decision. The design in this thesis is organised around these three jobs.

The coding stage also connects to both ends of the pipeline. It reads the outputs of the design stages, and its branch is what the Security and QA agents scan and test. When the Security Agent finds a vulnerability, the fix comes back to the Coder Agent as a revision, so the agent must also be able to make small, targeted changes to code it wrote earlier.

1.3 Problem statement

When an LLM-based agent writes code, three problems are common. First, it drifts from the plan: it adds files nobody asked for, leaves planned files out, or invents its own structure. Second, its own account of what it did is unreliable. An agent may report success when the project does not build, or may make an error go away by switching off the check that found it. Third, its changes are hard to review and undo when they are written straight into a working project with no record of what changed.

Locally hosted models make these problems worse, because some of them cannot reliably use tools at all, and the ones that can still run out of steps on larger features. The problem this thesis addresses is therefore: how can the coding stage of an LLM-based pipeline implement approved designs faithfully, prove with evidence that the result builds and runs, and present the change so that a person can approve or reject it safely?

1.4 Research gap

Coding agents such as SWE-agent [3], OpenHands [4], CodeAct [5], HyperAgent [6] and Agentless [7] have shown strong results on resolving issues in existing repositories, typically measured on SWE-bench [8] by whether the project's tests pass afterwards. AgentCoder [9] adds generated tests to the loop. These systems start from an issue, not from a reviewed design, and they are evaluated after the fact rather than paused for a person to approve each change.

Takerngsaksiri et al. [10] show that engineers value reviewing an agent's plan and its code. Surveys of LLMs in testing [11] and security [12] point out that most systems still rely on the model's own judgement of its output. What is missing is a coding stage that is driven by approved design artefacts, checks its plan against them before writing code, verifies the result with deterministic checks in an isolated environment, and leaves the final decision to a person looking at the real change.

1.5 Aim and objectives

The aim of this work is to design, implement and evaluate the Coder Agent of AutoForge, so that the code it produces follows the approved design, is verified by evidence rather than self-report, and reaches the main codebase only with human approval.

1. O1: Produce a code plan from the approved artefacts and validate its coverage of endpoints, data entities and functional requirements before any code is written.
2. O2: Implement the plan on an isolated git branch with a tool-using coding loop, with a non-agentic path for models that cannot use tools.

3. O3: Verify every attempt in a Docker sandbox with build, boot and deterministic checks, feeding failures back into the next attempt.

4. O4: Present the change through artefacts built from the real git diff, merge only on approval, and evaluate the agent on tests, the team's case study and stored runs.

1.6 Research questions or hypotheses

1. RQ1: Does validating the code plan against the approved artefacts keep implementation aligned with the design?
2. RQ2: Can sandboxed, deterministic verification reliably distinguish code that builds and runs from code that does not, and what does it miss?
3. RQ3: Does retrying with the previous failure as feedback lead to verified code within a small number of attempts?
4. RQ4: Does reviewing the real diff before a git merge catch problems that verification does not?

1.7 Scope limitations and assumptions

The Coder Agent targets one stack: a Next.js App Router application in TypeScript with MongoDB through Mongoose, started from a fixed scaffold that the workspace service creates for each project. It implements one feature at a time on its own branch. It does not write automated tests, which is the QA Agent's job, and it does not run a security scan, which is the Security Agent's job, although it reuses the same sandbox.

The evaluation is small. The team's case study covers three features, and the stored-run analysis covers nine code versions from three features in two projects. Most runs used one default model. This is enough to show how the mechanisms behave, including how they fail, but not to give general rates.

I assume Docker is available on the machine that runs AutoForge, that the sandbox can reach the npm registry to install packages, and that a person with enough knowledge of web development reviews each change.

1.8 Individual contribution

Within the team, my responsibility was the coding stage. My contributions are:

- the code planner, the plan validator and the re-planning loop that feeds coverage gaps back to the model;
- the coding loop's tool set, scoped to the project workspace, and the batch coding path with automatic detection of whether a model supports tool calls;
- the workspace service's per-project git repository, Next.js scaffold, feature branches, merge, discard and undo-merge;
- the Docker sandbox service and the verifier, with its build, boot and deterministic checks;
- the diff builder, merge report and requirement-to-code map, the revision flow, and the live preview; and
- the automated tests for these modules, and my share of the team's evaluation and paper.

1.9 Thesis structure

Chapter 2 reviews related work and builds a conceptual framework. Chapter 3 describes the research method. Chapter 4 presents the design of the agent, and Chapter 5 covers implementation, integration and testing. Chapter 6 reports the results, including a close look at stored runs, and Chapter 7 discusses them. Chapter 8 considers commercialisation and sustainability, and Chapter 9 concludes.

2. LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

I began with two surveys, Hou et al.'s systematic review of LLMs for software engineering [13] and He, Treude and Lo's review of LLM-based multi-agent systems for software engineering [14], and followed their references to the systems most relevant to code generation. The review then focused on four themes: coding agents and how they act on a repository, how their output is evaluated, testing and security of generated code, and human oversight. The sources are those of the team's research paper; the aim was to place my component precisely, not to survey the whole field.

2.2 Domain and theoretical foundations

2.2.1 Tool-using coding agents

A tool-using agent works in a loop: the model decides on an action, such as reading a file, writing a file or running a command, the system carries it out, and the result goes back to the model for its next decision. Wang et al.'s CodeAct [5] argues that letting the model express actions as code makes such agents more capable, and Yang et al.'s SWE-agent [3] shows that the design of the interface between agent and computer, which tools exist and what they return, matters as much as the model. Both ideas shaped my tool set: tools are narrow, scoped to one workspace, and always return a readable message, even on failure, so the model can react instead of crashing.

2.2.2 Verification by execution

The most convincing evidence that code works is running it. SWE-bench [8] evaluates agents by running the repository's tests after the agent's change, and AgentCoder [9] puts test execution inside the generation loop. For a newly generated feature there are usually no tests yet, so verification has to start more simply: does the project install, compile, type-check and start, and do the specific things the plan promised actually exist? Running these checks in a disposable container keeps the host machine safe from whatever the generated code does.

2.2.3 Version control as a safety mechanism

Git branches let a change be developed in isolation and either merged or thrown away as a unit, and the diff between a branch and the main line is an exact record of what

changed. For an agent, this provides two things: a clean way to undo a rejected change, and a source of truth about what the agent did that does not depend on the agent's own account.

2.2.4 Human oversight of agent changes

He, Treude and Lo [14] list human-agent collaboration and verifiability among the open problems of multi-agent software engineering. Takerngsaksiri et al. [10] describe HULA, deployed in a large software company, where engineers review an agent's plan and its code; they report that engineers valued this involvement, while the quality of the generated code remained a challenge. Pelluru [15] discusses frameworks such as LangGraph for building multi-agent systems that can pause and resume, which is what makes a review gate practical.

2.2.5 Design science

My work builds an artefact and evaluates it in its intended setting, which places it within the design-science paradigm of Hevner et al. [16]: the artefact must address a real problem, be evaluated rigorously, and add knowledge about the design, including its limits.

2.3 Related research

SWE-agent [3] gives a model a small set of commands for viewing, searching and editing files, and shows that this agent-computer interface strongly affects success. OpenHands [4] provides an open platform in which agents edit code, run commands in a sandbox and browse the web. HyperAgent [6] divides the work among planner, navigator, editor and executor agents. Agentless [7] takes the opposite position and shows that a fixed, non-agentic pipeline of localisation, repair and validation can be competitive with agents at a fraction of the cost. My component sits between these: a fixed outer pipeline of plan, code, verify and review, with an agentic loop only for the step that needs it, and a non-agentic path when the model cannot use tools.

RepoCoder [17] retrieves code from the same repository to guide completion, which supports giving the coding loop tools to search the project and read the approved page designs instead of putting everything in the prompt. ChatDev [1] and MetaGPT [2] generate whole small applications through role agents. MetaGPT's engineer works

from a design document, which is close to my setting, but its review is done by other agents and the result is not verified in an isolated build before a person sees it.

Surveys of LLMs for testing [11] and for security [12] add a caution that applies directly to code generation: model output should be checked by something other than the model. Zhu et al. in particular recommend combining static analysis with LLM review. In AutoForge the Security Agent does this, and my verifier adds a small set of deterministic static checks of its own, such as refusing a hardcoded database connection string.

2.4 Existing systems products and techniques

Table 2.1 compares the systems most relevant to my component. N/R means the property is not reported in the publication.

Table 2.1: Coding-stage properties of related systems

System	Starts from	Verification	Isolation of changes	Human gate
SWE-agent [3]	Issue in a repository	Tests run after the run	N/R	N/R
OpenHands [4]	Task in a repository	Agent runs commands in a sandbox	Sandbox	N/R
Agentless [7]	Issue in a repository	Reproduction and regression tests	N/R	N/R
AgentCoder [9]	Function description	Generated tests in the loop	N/R	N/R
HULA [10]	Issue in a tracker	N/R	N/R	Yes (plan and code)
AutoForge Coder (this work)	**Approved plan and page designs**	**Sandboxed build, boot and static gates**	**Git feature branch + Docker**	**Yes, before merge**

In industry, AI coding assistants inside editors and hosted app generators are widely used. They are fast, but they usually rely on the developer to build and test the result, and on the developer's own version control habits to keep changes reviewable. The techniques my component combines, a validated plan, a scoped tool set, sandboxed verification and a branch per feature, are each familiar; what this work adds is their combination inside a human-gated pipeline, and an honest account of where the combination still falls short.

2.5 Gap synthesis and conceptual framework

The review shows that coding agents are usually driven by issues rather than approved designs, judged by tests after the run rather than verified before a person decides, and rarely isolate their changes in a way that makes a reject decision clean. Figure 2.1 shows the conceptual framework I used. Approved design artefacts and the project's repository go through three mechanisms, each paired with a control, to three outcomes. As elsewhere in AutoForge, the model proposes and the controls decide.

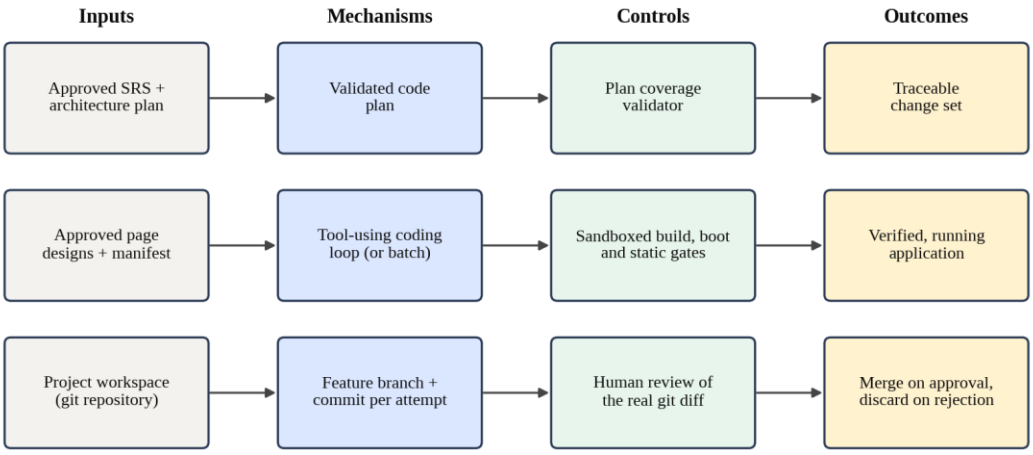


Figure 2.1: Conceptual framework for the coding component

3. RESEARCH METHODOLOGY

3.1 Research design

This is design-science research in the sense of Hevner et al. [16]. I built an artefact, the Coder Agent with its workspace, sandbox and verifier, to address the problem described in Chapter 1, and evaluated it inside the pipeline it was built for. Existing coding benchmarks such as SWE-bench [8] measure issue resolution in existing repositories, which is a different task from building a new feature from approved designs, so the evaluation combines automated tests, the team's case study, and an analysis of the runs the agent stored on real features.

Development followed build-and-evaluate cycles between June and August 2026. Many of the verifier's checks were added in response to a specific failure seen in a real run, for example a create form that could never save because the model required a field the form did not collect, or a home page that never linked to the new feature. I report these as findings, because the history of the checks is part of the answer to what verification needs to cover.

3.2 Data participants or study materials

The study materials were the approved artefacts given to the agent, the generated project repositories, and the artefacts the agent stored for each run: the code plan, file tree, code manifest, requirement-to-code map, setup instructions and merge report. Table 3.1 lists the runs analysed.

Table 3.1: Coder Agent runs analysed in this thesis

Feature	Project	Stored code versions	Source of evidence
Item Listing, Authentication, Shopping Cart	Sample E-commerce	As reported in the paper	Team case study
Add & List Items	Retail Store	1	Merge report, code plan, git history of the project repository
Item Listing	Finodil	2	Merge reports and code plans
Login and Signup	Finodil	6	Merge reports and code plans, including two security-driven revisions

The default model was qwen3-coder served through Ollama. The agent checks at run time whether the selected model supports real tool calls and chooses the coding path accordingly. No external participants took part; approvals were made within the project team, and no personal data was collected. Where a run used a real MongoDB database, the merge report records it; the connection strings themselves are kept in the project's local environment file and are not reproduced here.

3.3 Experimental or study procedure

Component testing. The planner's prompt construction, the plan validator, every tool, the workspace service, the verifier's checks, the diff builder and the revision helpers were tested against fixtures. Model calls were stubbed, and sandbox commands were replaced by recorded results, so that failure paths such as a loop hitting its step limit or a build failing on every attempt could be exercised deliberately.

Pipeline case study. As part of the team evaluation, each of three features was carried through all seven stages with human approval, and the team recorded the code versions, the decision at each gate and the reason for each revision.

Stored-run analysis. For the nine stored code versions in Table 3.1, I recorded the verification outcome, the result of every verification step, the number of coding attempts, and, where the step failed, the reason. For the retail store feature I also compared the merged code with the approved SRS, the architecture plan and the page designs, and examined the project's git history.

3.4 Measures and evaluation metrics

Table 3.2: Measures used for each research question

RQ	Measure	How it is obtained
RQ1	Plan coverage gaps found; planned versus touched files; requirements traced to files	Plan validator tests; code plans and requirement-to-code maps
RQ2	Verification outcome and per-step results; causes of failure; requirements not met despite a pass	Verifier tests; merge reports; comparison with the SRS
RQ3	Coding attempts used per version; outcome after retries	Merge reports
RQ4	Revisions requested at the coding stage and their causes; changes made after verification	Case-study records; git history

3.5 Data analysis

The analysis is descriptive. With nine stored versions and three case-study features, I report counts and individual cases rather than rates. Each finding is tied to a stored merge report, code plan or commit. Where a failure had an environmental cause, such as Docker not running, I report it separately from failures caused by the generated code, because the two say very different things about the agent.

3.6 Ethics privacy security and responsible AI

Running generated code is the main risk in this component, so all commands run inside a disposable Docker container with only the project's workspace mounted and a memory limit, never directly on the host. The coding loop's shell tool accepts only npm, npx and node, plus the read-only git status and git diff commands, and every path the tools accept is checked to stay inside the project's workspace.

Database credentials need care. A reviewer can give the agent a MongoDB connection string in a chat message; the agent saves it to the project's local environment file, removes it from the text before the text reaches the model, and never writes it into source code. The verifier fails any run that hardcodes a connection string. Finally, the agent never merges its own work. A person reviews the diff and the merge report, which states plainly whether verification failed, and only that person's approval merges the branch.

3.7 Reproducibility and research artefacts

All code, prompts and tests are in the team's repository, each generated project has its own git repository, and every run's artefacts are stored as numbered versions. Table 3.3 lists the settings that affect results.

Table 3.3: Configuration parameters of the component

Parameter	Value
Default LLM	qwen3-coder via Ollama
Planning attempts	Up to 4
Coding attempts	Up to 3
Coding loop step limit	100
Sandbox image	Custom image based on node:20, with git and a JavaScript parser
Sandbox memory limit	2 GB
Timeouts: install / build / boot / other scripts	600 s / 300 s / 60 s / 120 s

Parameter	Value
Generated stack	Next.js App Router, TypeScript, MongoDB with Mongoose, Tailwind CSS

Model output is not deterministic, so repeated runs produce different code. The plan validator, the verifier's static checks, the diff builder and the git operations are deterministic, and given the same repository they give the same result.

4. PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements for the Coder Agent came from the team's pipeline objectives, from what the Security and QA agents need from a coded feature, and from problems found in real runs. Table 4.1 lists the functional requirements and Table 4.2 the non-functional ones.

Table 4.1: Functional requirements of the component

ID	Requirement
C3-FR-01	Run only when an approved SRS and an approved architecture plan exist; use approved page designs when available
C3-FR-02	Produce a code plan listing every file to create, modify or delete, with the requirements each file serves
C3-FR-03	Check that the plan covers every planned endpoint, data entity and functional requirement before coding
C3-FR-04	Keep every project in its own git repository, created from a working Next.js scaffold
C3-FR-05	Develop each feature on its own branch and commit every coding attempt
C3-FR-06	Implement the plan with tools scoped to the project workspace, or one call per file for models without tool calls
C3-FR-07	Verify each attempt in a sandbox and feed the failure into the next attempt, up to a fixed limit
C3-FR-08	Build the file tree, code manifest, requirement-to-code map and merge report from the real git diff
C3-FR-09	Merge the branch into main on approval and discard it on rejection; allow an approved merge to be undone
C3-FR-10	Update the project manifest after a merge so that later features know what exists
C3-FR-11	Apply revision requests on top of the existing branch, including fixes requested by the Security Agent
C3-FR-12	Accept a database connection string from the reviewer without exposing it to the model or the code
C3-FR-13	Start and stop a live preview of the feature on request

Table 4.2: Non-functional requirements of the component

ID	Quality	Requirement
C3-NFR-01	Safety	Generated code must only run inside a disposable container with limited memory and time
C3-NFR-02	Containment	Tools must not read, write or execute outside the project workspace
C3-NFR-03	Honesty	Reported results must come from real commands and the real diff, never from the model's account
C3-NFR-04	Reversibility	Any change must be removable as a unit, before or after merging
C3-NFR-05	Robustness	A failed attempt, a crash in the loop or an unavailable sandbox must produce a reviewable result, not a lost run
C3-NFR-06	Portability across models	The agent must work with models that cannot use tools
C3-NFR-07	Responsiveness	Progress, including each tool call, must stream to the interface

4.2 Component architecture within the complete solution

Figure 4.1 places the component in the complete system. The orchestrator runs the Coder Agent once the architecture plan and page designs are approved. The agent uses three services: the workspace service, which owns each project's git repository; the sandbox service, which runs commands in Docker; and the preview service, which starts the built application in a container so a reviewer can try it. The verifier sits between the agent's coding attempts and the reviewer.

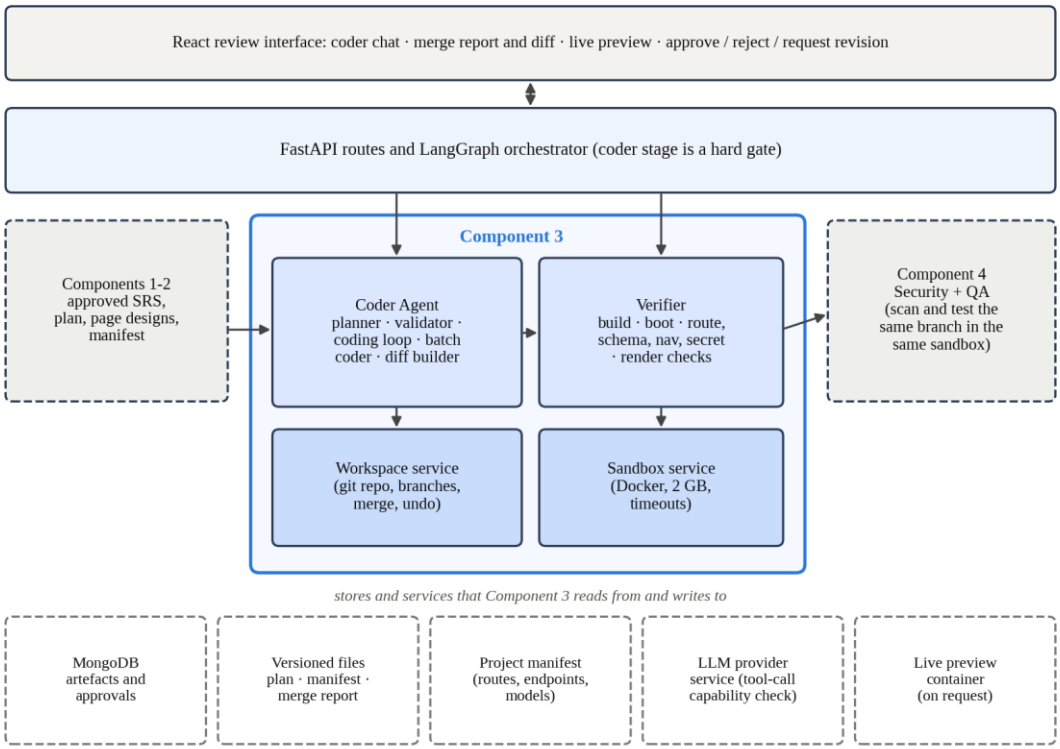


Figure 4.1: The component's place in the complete AutoForge architecture

The boundary with the earlier stages is the approved artefact: the SRS or Enhanced SRS, the architecture plan with its implementation plan, and the UI/UX integration manifest and page designs. The boundary with the later stages is the feature branch: the Security and QA agents scan and test exactly the code the reviewer will be asked to approve, in the same sandbox.

4.3 Design alternatives and decisions

Table 4.3: Main design decisions

Decision	Alternative	Reason for the choice
Plan first, then code	Let the loop decide what to build as it goes	A plan can be validated against the design in seconds, before expensive coding starts
Fixed scaffold per project	Let the model create the project	The app always has a working base, so build and boot failures point at the feature, not the setup
Branch per feature, merge on approval	Write directly to main	A rejected change is discarded cleanly and an approved one can be undone
Verify with real commands in Docker	Trust the loop's report; run on the host	Evidence instead of self-report, without risk to the host

Decision	Alternative	Reason for the choice
Hard gates for build, boot and specific static checks	Gate on everything, including lint and heuristics	Cosmetic or heuristic failures would block correct work from a fragile local model
Batch path for models without tool calls	Refuse to run such models	The team's models differ; the stage should still produce reviewable code
Reports built from git	Ask the model to summarise its changes	The diff is the only account that cannot be wrong
Proceed after the last failed attempt, clearly marked	Retry indefinitely or discard the work	A person can judge a near-miss; the merge report makes the failure impossible to miss

The decision about lint is worth explaining, because our research paper lists lint among the sandbox steps. The scaffold has a lint script, but the verifier deliberately does not run it as a gate. ESLint flags many real but harmless issues, such as unused imports, and with a local model these would have failed many runs that were otherwise correct. A reviewer can still run lint themselves. The build step already includes Next.js's type check, which catches the errors that matter most.

4.4 Detailed design

4.4.1 Planning and plan validation

The planner makes one model call that reads the approved SRS (or Enhanced SRS), the architecture plan, the UI/UX integration manifest and the project manifest, and returns a code plan. The plan has a summary, a list of files, any new npm dependencies and any environment variables needed. Each file entry gives a path, an action (create, modify or delete), a rationale and a maps_to list naming the endpoints, entities and requirement identifiers it serves.

The plan validator then checks the plan against the design. The files list must not be empty, every entry needs a path, a valid action and a maps_to list, and between them the files must cover every API endpoint and data entity in the architecture plan and every functional requirement in the SRS. If anything is missing, the validator's message and the previous plan are sent back with an instruction to fix only those gaps. This happens up to four times. The limit was raised from two after a real run showed the model closing the gaps step by step but being cut off before it finished.

4.4.2 Workspace and branches

Each project has its own git repository. When it is first created, the workspace service writes a complete Next.js App Router scaffold: package.json, TypeScript, Tailwind and ESLint configuration, a root layout and home page, a health route at /api/health, a MongoDB connection helper and seed data, and an example environment file. Because the scaffold builds and starts on its own, a failed build after a coding attempt points at the feature, not at the project setup.

Every feature is developed on its own branch created from main. Each coding attempt is committed, so the branch history shows exactly what each attempt changed. On approval the branch is merged with a no-fast-forward merge, which keeps the feature as one identifiable unit in main's history; on rejection it is discarded. An approved merge can also be undone, which the approval service uses when an approval is revoked.

4.4.3 Coding loop and batch path

Figure 4.2 shows the whole flow. Before coding, the agent checks whether the selected model really supports tool calls. OpenAI and Anthropic models are treated as supporting them; for an Ollama model, the agent asks the Ollama server, which reports each model's capabilities. This check was added after a model wrote its tool calls as plain text instead of real tool calls, which caused a confusing failure.

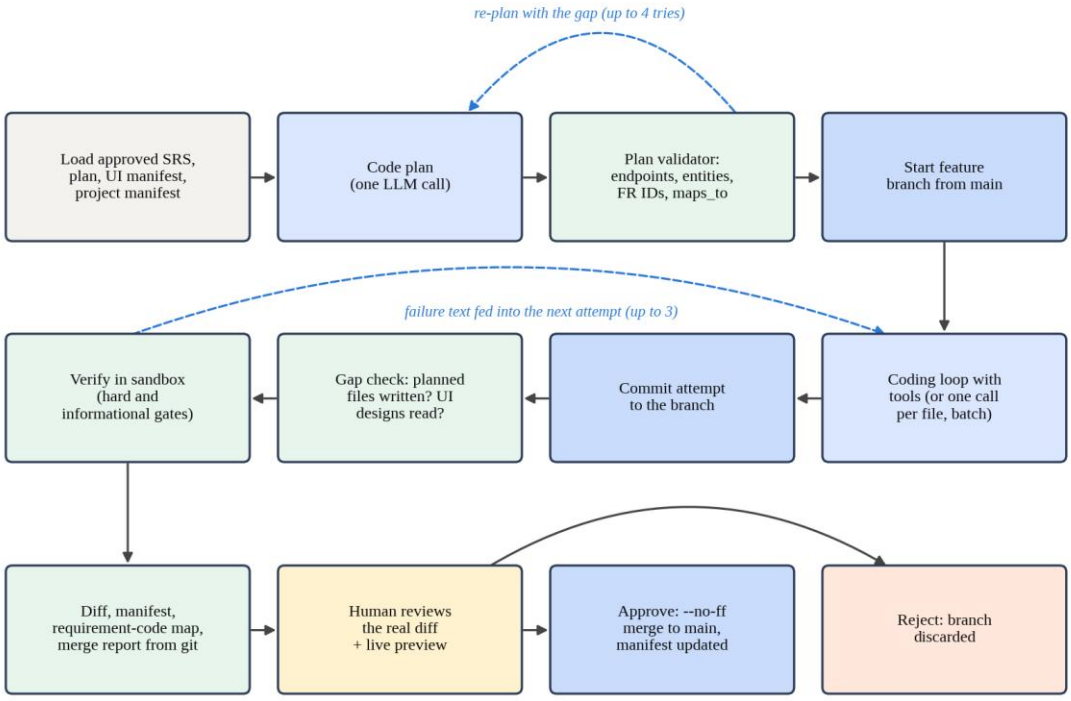


Figure 4.2: Coder Agent flow from plan to merge

With a tool-calling model, the plan becomes the task for an agentic loop of up to 100 steps. The loop has thirteen tools, listed in Table 4.4. Every path is resolved inside the project workspace and rejected if it would escape it, and every tool returns a plain message even when it fails, so the model can react to the problem. The apply_patch tool replaces text only when the text to find occurs exactly once, which prevents an ambiguous edit.

Table 4.4: Tools available to the coding loop

Tool	Purpose
list_dir, read_file, search_code	Explore the workspace
write_file, apply_patch	Create a file, or replace one exact piece of text
run_shell	Run npm, npx or node, or git status and git diff, in the sandbox
check_syntax	Parse a file with a JavaScript parser before moving on
check_component_styling	Find pages and components with no Tailwind styling
read_project_manifest	See the routes, endpoints and models already built
read_ui_component_design, read_ui_page_design	Read the approved HTML design of a component or page
list_unread_ui_designs, list_unimplemented_planned_files	Check what is still left to read or write

With a model that cannot use tools, the batch path makes one call per planned file and writes the returned content. One call per file, rather than one call for the whole plan, is a deliberate choice: a real feature's files sent back in a single response exceeded the configured output limit and were cut off mid-file. The batch path has no way to read other files or check syntax while it works, so it is expected to be less reliable, but it lets the stage produce reviewable code with any model.

4.4.4 Attempts, gap checks and verification

Each attempt ends with a commit, even if the loop crashed or reached its step limit, so partial progress is never lost. The agent then checks two gaps before verifying: whether every planned file was actually written, and whether the loop read the approved page designs before writing the related front-end files. If either gap exists, the next attempt is told exactly what is missing.

Verification runs inside the sandbox and is summarised in Figure 4.3. Ten checks are hard gates: if any fails, the attempt fails. The first four prove the application works at all: next.config.mjs must not switch off type or lint errors, which is the easiest way for a struggling model to make a build pass; the dependencies must install; next build must compile and type-check; and the built app must start and answer at /api/health. The next six check specific promises: every planned endpoint has a Route Handler that exports the right method; every Route Handler that connects to the database handles the case of no database, so the app serves seed data instead of crashing; no MongoDB connection string is hardcoded; every required field in a Mongoose schema can be set by some planned front-end file; every new page is linked from somewhere; and the home page renders in a real browser.

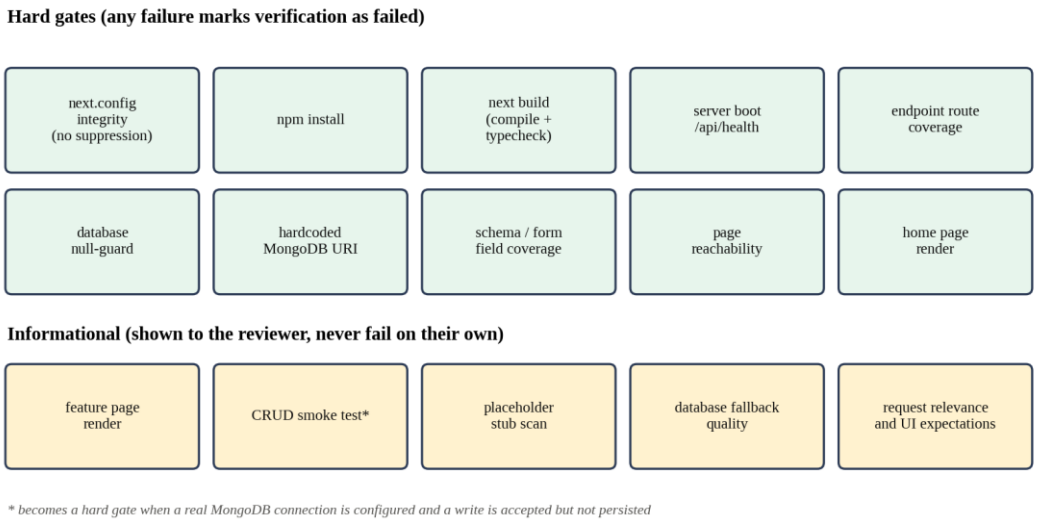


Figure 4.3: Hard and informational verification steps

Five further checks are informational. They appear in the merge report for the reviewer but do not fail the attempt on their own, because they are heuristics: whether feature pages render, a create-then-read smoke test against the running app, a scan for placeholder logic, a check of what the app returns when no database is configured, and word-overlap checks between the diff and the reviewer's request or the SRS's UI expectations. The smoke test becomes a hard gate in one case: when a real database is configured and a write is accepted but never appears on a read, which is clear evidence that a route is not using the database.

If verification fails, its output becomes part of the next attempt's instructions. After three failed attempts the agent stops and saves the result with verification marked as failed. It does not keep retrying, and it does not hide the failure; the merge report states it in its first line.

4.4.5 Reporting, approval and merge

After coding, the diff builder compares the feature branch with main and produces the file tree, the code manifest, the requirement-to-code map, setup instructions and the merge report. The requirement-to-code map starts from the files that were really changed and joins them with the plan; a changed file that was not in the plan is still listed, marked as unplanned, rather than disappearing. The merge report lists every

verification step with its result and output, the files added, modified and deleted, and the diff.

Approving the code diff of a version, which carries the file tree and merge report, merges that version's branch into main and updates the project manifest with the feature's routes, endpoints and models. Rejecting it discards the branch. Only a person can trigger either.

4.4.6 Revisions, security fixes and the live preview

A revision builds on the existing branch rather than starting again, because it is a targeted change to work that has already been verified. When the reviewer's comment names a real file of the feature, a single planning call is enough. When it does not, a short read-only exploration lets the planner look at the code before deciding which files to change; keyword matching against file names and contents provides hints. The fast path exists because a request as small as fixing a typo in a named file once took more than an hour under an exploration-for-everything design.

Security fixes arrive the same way: the Security Agent's findings become a revision comment. An informational check then confirms that every file named in the findings was actually touched. A reviewer can also paste a MongoDB connection string into a comment. The agent stores it in the project's local environment file, removes it from the comment, and, if the comment contained nothing else, simply restarts the live preview instead of running a whole coding cycle. The live preview itself is started and stopped only on request, runs the built application in a sandbox container, and is cleaned up if the backend restarts.

4.5 Standards and professional practice

The design follows ordinary professional practice for changes to a codebase: one branch per change, a commit per step, review of the diff before merging, and merges that keep the change identifiable. The generated project follows Next.js App Router conventions and TypeScript type checking, and the verifier refuses the common shortcut of suppressing type errors in the configuration.

On security, the design applies least privilege to the agent's tools, runs untrusted code in a container, keeps secrets in environment files rather than source code, and refuses

code that hardcodes a database connection string, which corresponds to the Common Weakness Enumeration entry for hardcoded credentials (CWE-798). Model output is treated as untrusted input throughout.

5. IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

Table 5.1 lists the main modules and their size. The agent itself is large mainly because it supports two coding paths, run and revise, and a streaming version of each.

Table 5.1: Technology stack used by the component

Concern	Technology	Why it was used
Agent backend	Python, FastAPI, Pydantic	The team's shared backend; typed schemas and streaming
Coding loop	LangChain ReAct agent on LangGraph	Tool-using loop with a step limit and streamed tool activity
Default LLM	qwen3-coder via Ollama	Local model with real tool-call support
Repositories	Git through GitPython	Branches, commits, diffs and merges from Python
Sandbox	Docker (custom node:20 image)	Disposable, memory-limited execution of untrusted code
Generated app	Next.js App Router, TypeScript, Tailwind CSS	One framework for pages and API routes; type checking in the build
Generated data layer	MongoDB with Mongoose	Matches the team's stack and the architecture plans
Page render checks	Playwright headless Chromium against next start	Proves pages load in a real browser, not only that they compile
Testing	pytest	Fixture-based tests with stubbed models and sandbox results

5.2 Component implementation

Table 5.2 lists the main modules and their size. The agent itself is large mainly because it supports two coding paths, run and revise, and a streaming version of each.

Table 5.2: Main modules of the component

Module	Lines	Responsibility
coder_agent/agent.py	2,529	Run, revise and streaming flows; retries; gap checks; artefacts; merge and discard
coder_agent/prompt.py	993	Planner, coding, repair and revision prompts
coder_agent/verify.py	764	Sandboxed verification and assembly of all steps
Ten checker modules	1,331	Route, schema/form, navigation, render, secret, database, functional, UI and security-coverage checks
coder_agent/tools.py	616	Workspace-scoped tools for coding and revision planning
coder_agent/planner.py, plan_validator.py	453	Code plan generation and coverage validation
coder_agent/batch_coder.py, coding_loop.py	296	Non-agentic and agentic coding paths
coder_agent/diff_builder.py	177	File tree, manifest, requirement map, merge report

Module	Lines	Responsibility
services/workspace_service.py	1,770	Repositories, scaffolds, branches, merge, undo, environment files, exports
services/sandbox_service.py	245	Docker commands and background services
services/preview_service.py	284	Live preview start, stop and clean-up

5.2.1 Agent orchestration

The run function loads the approved SRS, the Enhanced SRS when approved, the architecture plan and, if one exists, the UI/UX integration manifest, together with the project manifest. It then plans with retries, stops any running preview for the feature, starts the feature branch, records the starting commit, and hands over to a single dispatch function that chooses between the agentic and batch paths. The same dispatch is used by revisions, so both paths behave identically from the agent's point of view.

Inside the agentic path, two kinds of failure are caught and turned into failed attempts: the loop reaching its step limit, and any unexpected error raised inside the loop. Before this was done, a revision could crash in the middle of an attempt and leave no saved artefact and an uncommitted workspace, which looked the same to a reviewer as the agent doing nothing. Now whatever was written is committed and the next attempt is told what is still missing.

The streaming versions send an event for every tool call, labelled in plain words such as "Reading app/page.tsx" or "Running: npm run build". The coding loop otherwise produces no text of its own for long periods, and without these events a reviewer watching the chat would see a single static message for the whole attempt.

5.2.2 Tools and sandbox

Every tool is built for one project and one feature, so the model never passes a project identifier and cannot address another project. The shell tool splits the command and allows only npm, npx and node, or git with the status or diff subcommand; anything else is refused with a message. Commands run through the sandbox service, which starts a container from the custom image with the workspace mounted, a 2 GB memory limit and a timeout, and returns the exit code and output as data. A timeout is reported with the conventional exit code 124 and a note in the output. If Docker itself cannot be reached, the service returns a failure result saying so instead of raising an error, which is why such runs still produce a merge report.

The image is based on node:20 rather than the smaller node:20-slim. The slim image has no git binary, which silently broke the git status and git diff checks the coding prompt encourages, and the custom image also includes a JavaScript parser so that check_syntax works before a project has installed its own packages. The memory limit was raised to 2 GB after builds were killed by the old limit with a bare exit code 137 that gave no hint of the cause.

5.2.3 Verifier and checkers

The verifier reads package.json to decide what to run, rather than asking the model, and runs the steps described in Section 4.4.4 in a fixed order. The application-level steps call the sandbox; the static checks read the workspace directly. Each checker is a small module that returns a list of problems, which the verifier turns into a step with a status of passed, failed, skipped or info and a readable output.

Several checkers record the incident that led to them. The schema and form check was added after a live "Failed to save item" bug: the model's Mongoose schema required a unique id field that the create form never set, so the first item saved and every later one collided. The navigation check was added after every feature built in an early project added a route but no link, so the home page never changed. The hardcoded-URI check followed a Security Agent finding of a hardcoded connection string (CWE-798) in generated code.

5.2.4 Workspace and diff builder

The workspace service creates the scaffold, starts and resumes feature branches, commits, computes the diff against main and the list of files touched since a given commit, merges, discards, undoes merges and exports a feature's code as a zip file. It also keeps an older Express and Vite scaffold for two projects created before the move to Next.js, and detects which stack a repository uses so the two are never mixed. When the scaffold improves, existing repositories are upgraded by adding the missing files rather than being recreated.

The diff builder is pure code. It reads the diff and the plan and produces the artefacts in Section 4.4.5. The merge report includes the verification steps, the changed files and a truncated line-by-line diff, and states at the top whether verification passed, how

many attempts were used, and whether a real database connection was used during verification.

5.3 Integration with the other components

With the design stages, the integration is through the implementation plan, the integration manifest and the approved page designs. The coding loop can read each approved component or page design as HTML, and the gap check makes sure it has done so before writing the related front-end files. With the requirement stage, the integration is through the SRS, which the planner reads and which the informational UI expectations check compares with the touched files.

With the Security and QA agents, the integration is the feature branch and the shared sandbox: both run their commands through the same sandbox service against the same workspace, so they test exactly what the reviewer will approve. Security findings come back as revisions, and the revision flow resolves file names mentioned in the findings against the files the feature has touched. After a merge, the project manifest is updated with the feature's routes, endpoints and models, which the Architecture Agent reads when planning the next feature.

5.4 Verification and software testing

Table 5.3 summarises the automated tests. Appendix A lists every file.

Table 5.3: Automated tests for the component

Area	Test files	Test functions
Agent flow, retries, revisions, prompts, model capability	12	167
Plan validator and diff builder	2	19
Verifier and checkers	13	114
Coding tools	1	35
Workspace, scaffold, merge and preview	8	75
Total	**36**	**410**

The tests focus on the paths that run only when something goes wrong, since those are hardest to see in normal use: a loop that hits its step limit, an exception inside the loop, a commit that itself fails, verification failing on every attempt, Docker being unavailable, and a model without tool support. The tool tests check that paths outside the workspace and disallowed commands are refused. The verifier tests check each

gate in both directions, for example that a page with no link fails reachability and a linked one passes.

The last recorded run of the backend suite, on 31 August 2026, collected 462 test cases in these areas, counting parameterised cases. The failures recorded for the component in that run's cache all belong to tests that have since been renamed or removed, several of them from the earlier Express and Vite design, so no current test is recorded as failing. As with the rest of the backend, some tests need a running MongoDB database through the shared test setup.

5.5 Deployment and operation

The component needs Docker on the machine that runs the backend, with the custom sandbox image built once from the repository's Dockerfile, and network access from the sandbox to the npm registry. Generated projects live in a workspaces folder, one repository per project, and persist across restarts. A project's database connection, when provided, is kept in that project's .env.local file, which the scaffold's .gitignore excludes from version control.

6. RESULTS AND VALIDATION

This chapter reports the results in the order of the research questions, after summarising what the automated tests established. Pipeline-level results come from the team's evaluation in our research paper. The rest come from nine code versions the agent stored for three features in two development projects, and from the git history of one of those projects.

6.1 Research results

Table 6.1 lists the behaviours the rest of the pipeline relies on and the tests that check them. All were passing in the last recorded run.

Table 6.1: Key behaviours confirmed by automated tests

Behaviour	Test file	Result
A plan that misses an endpoint, entity or FR is rejected with the gap named	coder_plan_validator	Pass
Paths outside the workspace and disallowed commands are refused	coder_tools	Pass
A loop hitting its step limit, or raising, becomes a committed, failed attempt	coder_agent_retries	Pass
After the last failed attempt, output is saved with verification marked failed	coder_agent_retries	Pass
A model without tool calls is routed to the batch path	coder_agent_batch_generation	Pass
Each verification gate fails and passes on the right input	coder_verify and checker tests	Pass
A write that is accepted but not persisted fails when a real database is set	coder_verify_mongodb_gates	Pass
Unplanned changed files appear in the requirement map, marked unplanned	coder_diff_builder	Pass
A connection string is saved, stripped from the comment and never sent to the model	coder_env_uri	Pass
An approved merge can be undone	workspace_undo_merge	Pass

6.2 Product and technical results

6.2.1 Plan validation and alignment with the design (RQ1)

Every stored code plan passed the plan validator before coding began, and every requirement-to-code map matched the plan's files: the only unplanned files in any version were a lock file and, in the retail store feature, a small shared type file, both correctly marked as unplanned. In this sense the code stayed within the plan.

The maps tell a less comforting story about what a maps_to entry means. In the retail store plan, FR-DOM-001, the Domain Agent's requirement to view an item's details, is listed against all five planned files, but the code contains no detail view. In the first Login and Signup plan, all eighteen functional requirements are listed against the data model file. The validator only checks that each requirement appears somewhere, so a plan can satisfy it by listing a requirement against files that do not implement it. The answer to RQ1 is therefore that plan validation keeps the change set aligned with the design in structure, which files exist and which endpoints and entities they cover, but cannot confirm that each requirement is implemented.

6.2.2 What sandboxed verification found (RQ2)

Table 6.2 shows the outcome of all nine stored versions.

Table 6.2: Verification outcome of the stored code versions

Feature and version	Attempts	Outcome	Failed hard gate and cause
Retail Add & List Items v1	3	Passed	None
Finodil Item Listing v1	1	Failed	Schema/form: description, price, quantity, category required but never set by a form
Finodil Item Listing v2	1	Failed	Same schema/form gap
Finodil Login and Signup v1	3	Failed	Install and build: Docker daemon not reachable
Finodil Login and Signup v2	3	Failed	Build: module 'bcryptjs' not found
Finodil Login and Signup v3	3	Failed	Build: '@/lib/mongodb' has no default export
Finodil Login and Signup v4	3	Passed	None
Finodil Login and Signup v5 (security fix)	3	Failed	Install and build: Docker daemon not reachable
Finodil Login and Signup v6 (security fix)	2	Passed	None

Three things stand out. First, every failure caused by the code was specific and actionable: a dependency used but never added, an import that did not match the scaffold's MongoDB helper, and a data model requiring four fields that no form collected. Each of these would have been found by a developer only on running the app, and in the last case only when trying to save a record. Second, the two runs where Docker was not running were reported as failed steps with the reason in the output, not

as crashes, so the reviewer could see immediately that the problem was the environment, not the code. Third, the Item Listing reports record only one coding attempt even though the limit is three; the stored artefacts do not show why the further attempts were not made, and I have not been able to establish the cause.

The retail store run shows what verification does not cover. It passed all ten hard gates. Table 6.3 compares the verified code with the approved SRS and the design artefacts.

Table 6.3: Verified retail store code compared with the approved artefacts

Approved artefact says	Verified code does	Caught by verification?
Image is a file upload; JPEG, PNG or WebP up to 5 MB (VR-001)	Takes an image URL as text; no type or size check	No
Price and quantity are numbers	Stored as strings in the Mongoose schema; checked as numbers in the route	No
Users can view an item's details (FR-DOM-001)	No detail view	No
Two pages, /add-item and /list-items (UI/UX design)	One page at /add-list-items (architecture plan)	No
The three SRS UI expectations	Step reported that no UI expectations were available	Not run

Most of these differences have a traceable cause in earlier stages. The architecture plan's fallback typed price and quantity as text, as the Architecture Agent's thesis describes, and the Coder Agent followed the plan. The route follows the architecture plan's single page rather than the UI/UX design's two pages, because the two design artefacts disagreed and the plan took priority. The image upload is a choice the model made itself. None of them breaks the build, boot or any static check, so none could be caught by verification. Among the informational steps, the create-then-read smoke test failed with HTTP 400 because its synthesised payload used the text "test" for price, which the route correctly rejected; this shows why that check is informational.

For RQ2, sandboxed verification reliably separated code that builds and runs from code that does not, and its static checks caught a real class of runtime failure before a person saw it. It does not, and cannot, check that the code does what the specification says.

6.2.3 Retrying with feedback (RQ3)

Of the nine versions, three passed. Two passed on the full three attempts and one on two attempts. In the retail store feature, the git history shows how the attempts divided the work. The first attempt wrote only the back end: the model, the route, the client functions and a type file. The planned page was missing, which is exactly what the gap check looks for, and the second attempt wrote the page and added a link on the home page. The third attempt adjusted the home page and the client functions and added the lock file, after which verification passed. For the Login and Signup feature, versions 2 and 3 used all three attempts and still failed on build errors, and further versions were produced until version 4 passed.

For RQ3, retrying with the failure as feedback did lead to verified code, but not reliably within a single run: a third of the versions passed, and for the more complex feature it took several human-requested revisions on top of the automatic retries. The two security-driven revisions, versions 5 and 6, show the loop with the Security Agent working: its findings, a hardcoded MongoDB fallback credential and a NoSQL injection risk in the login route, came back as a revision, and version 6 passed verification after two attempts.

6.2.4 Human review of the real diff (RQ4)

In the team's Item Listing case study, the Coder stage was approved at version 4. The reviewer's revisions removed an identifier that had been specified by hand, because MongoDB generates one automatically, and updated the model, the API and the front end together. This is the same kind of problem as the schema and form gap in Table 6.2: a data model requirement that the rest of the code could not satisfy. Across the pipeline, four of the seven Item Listing stages were revised, the coding stage among them.

The retail store repository shows the other side of human review. After the agent's third attempt was verified and its artefacts saved, two further commits were made on the feature branch before the merge, with messages that do not follow the agent's commit format: one split the feature into two pages and fixed how images were displayed, and one added client-side validation to the form. These appear to be manual edits. They addressed real gaps from Table 6.3, the page split and input validation, but they were

merged without the verifier being run again. Table 6.4 shows the team's comparison of the whole pipeline with human gates on and off.

Table 6.4: Pipeline with human gates on and off

Measure	Gates ON	Gates OFF
Successful pipelines	3/3	2/3
Average revisions	5.0	1.0
QA pass rate	91%	78%
Security issues	1	4
Escaped defects	1	5
Human review effort	36 min	0

For RQ4, the evidence is that reviewers did find and correct problems in the code that verification had passed. It also shows a gap in the process: a correction made by hand on the branch is not verified unless the agent is run again, so the merged code is not exactly the code the merge report describes.

6.3 User or stakeholder validation

No developers or businesses outside the project team used the Coder Agent. Reviews were done within the team, by people who knew the expected result. An outside developer reviewing an unfamiliar diff might rely more on the merge report and less on their own reading, which would make the gaps in Section 6.2.2 more dangerous.

Use within the team did shape the design. Revising code by prompt on top of the existing branch, rather than only getting one automatic attempt, was added at a team member's request, and the informational check that a security-driven revision touched every file named in the findings followed a report from a team member who thought fixes were being missed. Investigation showed that the agent had touched every named file, and that the repeated findings came from run-to-run variation in the Security Agent's AI review layer; the check was added so that this is confirmed on every run instead of by hand.

6.4 Integration results

The Coder stage ran in every pipeline run of the team's evaluation, and all three features completed all seven stages with gates on. The QA Agent's results on the coded features were 2 of 2 tests passing for Item Listing, 14 of 16 for Authentication and 18 of 20 for Shopping Cart, and the Security Agent found 0, 1 and 1 issues respectively. Table 6.5 shows the cost of each run.

Table 6.5: Execution performance of the full pipeline

Run	Total time (min)	LLM calls	Retries	Tokens
Item Listing	36.4	24	4	31,800
Authentication	52.7	31	6	42,600
Shopping Cart	56.8	34	7	47,900
Baseline	31.2	18	3	25,400

The logs do not separate the time of each stage, so I do not claim a figure for the Coder stage alone. The retail store's git history gives a rough sense of it: the scaffold was created at 10:32 and the three coding attempts were committed at 11:20, 12:09 and 12:57, so each attempt with its verification took roughly 50 minutes on that machine. Most of that time is spent installing packages and building, which every attempt has to repeat.

6.5 Summary of findings

Table 6.6: Summary of findings by research question

RQ	Finding	Strength of evidence
RQ1	Plans covered every endpoint, entity and FR, and code stayed within the plan; maps_to entries can claim requirements that are not implemented	Tests; nine stored plans and maps
RQ2	Verification caught real build and data-model failures and reported environment failures honestly; it cannot detect specification mismatches	Tests; nine merge reports; one detailed comparison
RQ3	Three of nine versions passed, two of them on the last attempt; complex features needed human revisions as well	Nine merge reports
RQ4	Reviewers corrected verified code; manual edits were merged without re-verification	Case study; git history

7. DISCUSSION

7.1 Interpretation of findings

The results support the central choice of this component: do not trust the agent's account, check its work with real commands and present the real diff. Every failure in Table 6.2 that came from the code was found by running it, and every one of them would have reached the reviewer as a working-looking change if the agent's own report had been used instead.

The results also draw a clear line around what this kind of verification can do. It answers "does it run?" and a handful of narrower questions the checks were written for, such as "can a form actually save this model?". It does not answer "is this what was asked for?". The retail store feature passed every hard gate while taking image URLs instead of uploads, storing prices as text and leaving out a requirement. Those problems are visible in a two-minute look at the running app or the diff, which is exactly why the human gate exists.

A second lesson is that the Coder Agent inherits the quality of its inputs. Several problems in the retail store code began in the architecture plan's fallback or in the disagreement between the architecture plan and the page designs. The agent did the sensible thing, following the plan, but it had no way to know that the plan was weak. A coding agent at the end of a pipeline is only as good as the artefacts it is given, and the human gates before it matter as much as the one after it.

Third, retries help but do not replace review. One third of the versions passed, and the passes came late, on the second or third attempt. The attempt history shows retries doing useful work, filling in a missing page or fixing a dependency, but also shows that three attempts are not always enough for a medium-sized feature on a local model.

7.2 Comparison with literature baselines and existing solutions

SWE-agent [3] and OpenHands [4] showed that the design of the agent's tools strongly affects results; my experience agrees. Several changes to the tool set, such as returning failures as messages, requiring an exact single match for a patch, and adding tools that

list unwritten planned files and unread designs, made the loop noticeably more reliable. Agentless [7] showed that a mostly fixed pipeline can rival an agent; the Coder Agent uses a fixed outer pipeline of plan, code, verify and review, and keeps the agentic part small. The batch path takes this further for models that cannot use tools.

Where SWE-bench [8] judges a change by tests that already exist, the Coder Agent builds new features with no tests yet, so it relies on build, boot and static checks instead and leaves test writing to the QA Agent. AgentCoder [9] puts generated tests inside the loop; doing the same here is a natural next step. HULA [10] found that engineers value reviewing an agent's plan and code. The Coder Agent supports that review with a report built from git rather than from the agent, which addresses the verifiability concern that He, Treude and Lo [14] raise.

7.3 Research contribution

This research contributes a validated approach for integrating code generation, automated verification, version control, and human oversight within an LLM-based software development pipeline. The main research contribution is a tested design for a coding stage in which generated code is not accepted based solely on the coding agent's self-reported success. Instead, the implementation is driven by approved design artefacts, developed in an isolated Git branch, verified through deterministic execution-based checks in a Docker sandbox, and presented to a human reviewer before merging. This addresses the identified research gap in which existing coding agents commonly operate from issues or prompts and rely on post-generation testing or model-based judgement rather than combining design alignment, isolated execution, and human approval within a single coding workflow.

A second research contribution is the identification and implementation of deterministic verification mechanisms that go beyond checking whether generated software simply builds and runs. The verifier checks properties such as the existence of planned API endpoints, consistency between database schemas and front-end forms, page reachability, safe database handling, and the absence of hardcoded database connection strings. These checks were grounded in actual failures observed during

development, including unsatisfied schema requirements, missing navigation links, incorrect dependencies or imports, and hardcoded connection strings.

A third contribution is empirical evidence regarding both the capabilities and limitations of sandboxed verification. The evaluation demonstrated that execution-based verification can identify concrete and actionable implementation failures and can distinguish code failures from environmental failures such as an unavailable Docker daemon. However, the evaluation also demonstrated that successful execution does not guarantee compliance with the original specification. For example, a feature passed the verification gates while still differing from the approved requirements in areas such as image handling, data types, and required functionality. Therefore, the research provides evidence that automated verification and human review address different aspects of code quality and should be used together rather than treating automated verification as a replacement for human judgement.

Finally, the research contributes an empirical basis for understanding retry-based code generation. Failed verification results were fed back into subsequent coding attempts, allowing the agent to correct issues such as missing dependencies and incorrect imports. At the same time, the results show that retries are not sufficient for all medium-sized features, reinforcing the importance of a final human review stage. The study therefore contributes practical evidence about how planning, verification, retry mechanisms, Git isolation, and human approval can be combined in an LLM-based software engineering workflow.

7.4 Product contribution

The primary product contribution of this research is the implementation of the Coder Agent component of the AutoForge multi-agent software development platform. The component provides an end-to-end mechanism for transforming approved software requirements, architecture plans, and UI/UX designs into reviewable application code. It includes code planning and validation, agentic and non-agentic coding paths, workspace management, Git-based feature branches, sandboxed execution, deterministic verification, revision handling, reporting, and human-controlled merging.

The product provides a plan validation mechanism that checks the proposed implementation before code generation begins. The generated code plan identifies the files to be created, modified, or deleted and maps each file to relevant endpoints, entities, and functional requirements. The validator then checks whether the plan covers the required endpoints, data entities, and functional requirements, with targeted re-planning when coverage gaps are identified.

Another product contribution is the isolated Git-based development and rollback mechanism. Each feature is implemented on a separate branch and each coding attempt is committed. This allows generated changes to be reviewed as a specific unit, rejected without modifying the main branch, or merged only after human approval. An approved merge can also be undone, providing reversibility for AI-generated changes.

The product also introduces a Docker-based sandbox verification system for executing and evaluating generated code. The verifier performs hard checks covering dependency installation, compilation, type checking, application startup, API availability, database handling, schema/form compatibility, navigation, and credential protection. Additional informational checks provide further evidence to reviewers without unnecessarily blocking the coding process.

A further product contribution is the evidence-based merge reporting system. Instead of relying on the coding agent's description of what it changed, the system generates the file tree, code manifest, requirement-to-code mapping, verification results, and merge report from the actual Git diff. This provides the reviewer with an auditable representation of the generated change and its verification status before approval.

The implementation was supported by a substantial automated testing suite covering agent execution, retries, revisions, planning, verification, coding tools, workspace management, merging, and preview functionality. The thesis reports 410 automated tests for the component, together with a three-feature case study and analysis of nine stored code versions.

Overall, the product contribution is a practical Coder Agent that changes AI-assisted code generation from a process primarily dependent on model self-reporting into a plan-validated, sandbox-verified, Git-isolated, and human-approved workflow.

However, the product is deliberately designed as decision support rather than a replacement for developers: the evaluation showed that automated verification can prove that code runs and satisfies selected deterministic checks, but it cannot guarantee complete compliance with the original specification.

7.5 Limitations and threats to validity

7.5.1 Threats to Validity

Internal validity. The stored-run analysis was done by a member of the team that built the agent. To reduce bias, every finding is tied to a stored file or commit, and weak results are reported alongside good ones. The git history shows commits, not who made them in practice, so the conclusion that two commits were manual edits rests on their message format and timing.

External validity. The runs cover three features in two development projects, plus the case study's three features, all small to medium e-commerce features on one stack and mostly on one model. Larger features, other stacks or stronger models may behave differently.

Construct validity. A passing verification is a measure of whether the code builds, boots and meets the specific checks, not of whether it meets the specification. I have been careful not to treat it as the latter.

7.5.2 Limitations

- Verification cannot detect differences from the specification, such as a URL field instead of a file upload.
- Plan validation accepts a requirement listed against a file that does not implement it.
- Changes made on the branch outside the agent are not re-verified before merging.
- Lint is not a gate, unlike the description in our research paper, so style and some correctness warnings reach the reviewer unflagged.
- Each attempt repeats a full install and build, which makes attempts slow on an ordinary machine.

- The UI expectations check reported no expectations for a feature whose SRS had three; the cause was not established.
- No developer outside the team reviewed the agent's output.

7.6 Implications

The findings of this study have several practical implications for teams developing or adopting AI coding agents. First, AI-generated changes should be isolated from the main codebase and presented to reviewers using evidence derived from the actual repository. In particular, maintaining a dedicated feature branch and providing the reviewer with the real Git diff and verification results reduces reliance on the coding agent's own description of its work and provides a more reliable basis for human decision-making.

Second, verification mechanisms should evolve from observed failure modes. When a recurring or significant failure is identified in generated code, it can be translated into a deterministic verification check that can be automatically applied to subsequent implementations. In this study, this principle was applied to checks for issues including unlinked pages, inconsistencies between database schemas and form fields, and hardcoded database connection strings. This provides a practical approach for gradually increasing verification coverage based on evidence from actual agent behaviour rather than attempting to anticipate every possible failure in advance.

Third, verification results should always correspond to the exact state of the code that is presented for merging. Any modification made after verification, whether generated by the agent or introduced manually by a developer, can invalidate the previous verification result. Therefore, changes made after the last successful verification should trigger verification again before merging. This is particularly important because the evaluation demonstrated that human review may identify and correct issues that are not detected by the automated verifier. Re-verifying the final branch state would ensure that the evidence presented in the merge report remains consistent with the code that is ultimately integrated.

Overall, the findings imply that reliable use of AI coding agents requires more than code generation and automated testing alone. A controlled workflow combining isolated changes, evidence-based verification, iterative failure feedback, and human approval provides a stronger basis for integrating AI-generated code while recognising the limitations of automated verification.

8. COMMERCIALISATION AND SUSTAINABILITY

8.1 Value of the Component

The team's common report argues that teams using AI to build software need to see and approve what the AI did. The Coder Agent is where that need is sharpest, because its output changes a running application. Developers already use AI to write code, but they do not fully trust it. In the 2025 Stack Overflow Developer Survey, 84% of developers used or planned to use AI tools, yet only 32.7% trusted the accuracy of the output. 66% named code that is "almost right, but not quite" as their biggest frustration, and 45.2% said that debugging AI-generated code takes too long [18].

The component's value is not that it writes code, which many tools do, but that every change arrives with evidence. The plan is checked against the approved design before any code is written. The code is written on its own git branch, verified in a Docker sandbox against ten hard gates, and described in a merge report built from the real diff. A single decision then merges the branch or throws it away, and an approved merge can be undone (Table 6.1). The evaluation shows that this evidence is useful. Verification caught real failures before any person saw them: a missing dependency, an import that did not match the scaffold, and a data model requiring four fields that no form collected (Table 6.2). When Docker was not running, the report said so plainly instead of crashing. For a small agency, this turns AI-written code from something checked line by line on trust into something with evidence attached.

Table 8.1 shows who would use the component and what each group gains

Table 8.1: Target users of the component

User group	Need	What the component provides
Developers in small agencies and teams	Routine feature code without losing control of the repository	Code that follows the approved plan, on its own branch, verified before review
Technical leads and reviewers	Know what changed and whether it builds before approving	A merge report from the real git diff, verification results and a requirement-to-code map
Store owners (the clients)	Changes that do not break the live store	Nothing reaches the main branch without a verified build and a human approval; merges can be undone
Teams without powerful machines	Verification without a local Docker set-up	A hosted sandbox in the paid packages (Section 8.3)

This evidence comes from our own study, and no developer outside the team has yet used the component (Section 6.3). The team is therefore surveying developers and agency staff, as described in Section 7.1 of the common report.

8.2 Target Users and Market Position

The first market is the one AutoForge targets: small e-commerce development teams and agencies, beginning in Sri Lanka, whose IT and business process management industry has more than 680 companies, over 700 active startups and about 175,000 professionals [19]. Many of these developers already pay for AI coding tools, which makes this the most crowded part of the market. Table 8.2 compares the component with the alternatives, using published prices in October 2026 converted at 330.43 rupees to the dollar [25].

Table 8.2: Alternatives for producing feature code

Alternative	Typical cost	Strength	Weakness compared with this component
Writing code by hand	Developer time	Full control and judgement	Slow for routine features
AI assistant in the editor (GitHub Copilot, Cursor)	USD 10 to 40 (about LKR 3,300 to 13,200) per user per month [21], [22]	Fast; works in the developer's own flow	The developer must build, test and track changes; no link to an approved design
Prompt-to-app builders (Lovable, Bolt)	USD 25 to 50 per user per month [23], [24]	Very fast first version	Little control over structure; hard to review or undo a change
Issue-resolution agents such as SWE-agent [3] and OpenHands [4]	Free (open source) plus model fees	Strong on existing repositories	Start from an issue, not a reviewed design; no approval gate before merge
This component (within AutoForge)	Included from the free Community edition	Plan checked against the design; sandbox-verified; git-isolated; merged only on approval	Slower; verification cannot confirm that the specification is met

The component does not try to beat editor assistants at speed. Its position is different: it is the coding step of a pipeline that starts from an approved specification and design, and it hands the reviewer evidence rather than a suggestion. Editor assistants help a developer type; this component lets a lead decide whether a whole feature should enter

the repository. The two can also work together, since a developer can still edit the branch with any tool before approving it.

8.3 Business Model and Feasibility

The component fits within the open-core model and the packages proposed in the common report (Table 7.5): a free self-hosted Community edition, Team seats at LKR 6,000 per month, Agency seats at LKR 10,000 per month, and enterprise and academic licences. In the Community edition, the component runs with a local model and Docker on the developer's own machine, so it costs the team nothing to serve. Table 8.3 shows the paid elements that depend directly on my work.

Table 8.3: Revenue elements that depend on this component

Revenue element	Price (common report)	What this component contributes
Hosted sandbox verification	Part of every hosted feature run in the Team and Agency packages; extra runs LKR 400 each	The ten verification gates run in a hosted container, so teams need no local Docker or powerful machine
Git hosting integration	Team and Agency packages	An approved branch becomes a pull request in the team's own repository instead of a local merge
Enterprise / on-premise licence	From LKR 1,500,000 per year	Code, branches and builds stay entirely inside the customer's network
Setup services	LKR 75,000 per engagement	Adapting the scaffold and checks to a customer's existing repository conventions

The main cost of this component is verification time, not model tokens. The retail store's git history shows that each attempt, with its package install and production build, took about 50 minutes on the development machine (Section 6.4), and a feature may need up to three attempts. In a hosted service this sandbox time is the largest part of the variable cost that the common report allows for each seat. Caching installed packages between attempts, and building only when files have changed, would cut it substantially, and is a precondition for a hosted product at the proposed prices. Model costs are small by comparison: a whole feature run costs nothing in API fees with the local model, and at most about LKR 160 on a hosted model.

Technically, the component is already feasible. It ran in every feature of the team's evaluation, and the git isolation, verification and merge-on-approval behaviour is

covered by automated tests. The results in Chapter 6 also show what must change before it can be sold, listed in Table 8.4.

Table 8.4: Work needed before a commercial release

Issue found	Evidence	Change needed
Each attempt took about 50 minutes, mostly install and build	Section 6.4	Cache packages between attempts; rebuild only when files change
Only three of nine stored versions passed verification	Table 6.2	Better use of failure feedback; report clearly when a run needs a human revision
A requirement-to-code map can claim requirements that are not implemented	Section 6.2.1	Check each claimed file for evidence of the requirement, or label the map as the plan's claim
Manual edits on the branch were merged without re-verification	Section 6.2.4	Re-run verification automatically on any new commit before a merge
Verification cannot check that the code meets the specification	Table 6.3	Present verification as build evidence only; rely on QA tests and the reviewer for behaviour

8.4 Sustainability and Maintenance

Scalability. Each feature already runs in its own workspace, on its own branch and in its own sandbox, which is the right shape for a multi-user service: runs from different customers never share files. To serve many teams, the hosted service would need a job queue, a pool of sandbox containers sized to the number of runs, and the package cache described above, so that cost grows with the number of feature runs rather than with the number of customers. The stack is fixed to Next.js, TypeScript and MongoDB. Supporting a second stack would need a second scaffold and a second set of checks; the workspace service's existing support for an older stack shows that this is feasible, but not free.

Intellectual property. The code generated for a customer belongs to that customer, and the terms of use must say so. Two further points need care before a commercial release. First, the generated applications install third-party npm packages, and the customer inherits their licences, so the merge report should list new dependencies and their licences for the reviewer. Second, a model can occasionally reproduce code it has seen during training, which is one more reason to keep a person responsible for every merge. The scaffold, checks and agent code remain the team's property, subject to the

university's intellectual property policy and to the licences of Docker, Node.js and the Next.js toolchain.

Compliance. Sri Lanka's Personal Data Protection Act, No. 9 of 2022, takes effect on 1 January 2027 [20]. The code this component writes is the code that stores customers' accounts, addresses and orders, so it carries much of a store's compliance risk. Two design choices already help. A MongoDB connection string supplied by a reviewer is saved for the workspace, removed from the comment and never sent to the model (Table 6.1), so credentials do not leave the customer's control. And security findings come back to the Coder Agent as a revision; in the Login and Signup feature, a hardcoded fallback credential and a NoSQL injection risk were fixed this way and the result passed verification (Section 6.2.3). A hosted service must add one rule: generation and verification run against test databases only, never against a store's live customer data.

8.5 Contribution to the Sustainable Development Goals

The common report links AutoForge to SDG 9 (Industry, Innovation and Infrastructure) and SDG 8 (Decent Work and Economic Growth). The Coder Agent's contribution to SDG 9, and in particular target 9.5 on upgrading technological capabilities, is a safer way to use AI for building software, with isolation, evidence and human approval, which small firms can run on their own hardware. Its contribution to SDG 8, targets 8.2 and 8.3, would come through productivity in small development firms, which could deliver routine features faster without giving up review.

These contributions are potential rather than demonstrated. There are also negative effects to manage. Repeated builds use energy; caching and stopping after a clear environmental failure, rather than retrying, would reduce the waste. Tools that write routine code may also reduce the practice junior developers get. The component keeps a developer responsible for every merge, and the review of a real diff is itself a place where junior developers can learn by reading and correcting code. A responsible next step would be a pilot with a real agency, measuring review time, accepted changes, defects found after merge and the energy used per feature.

9. CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis set out to make the coding stage of an LLM-based pipeline implement approved designs faithfully, prove with evidence that the result builds and runs, and present the change so that a person can approve or reject it safely. The Coder Agent plans the change and validates the plan against the design, codes on an isolated branch with tools scoped to the project, verifies each attempt in a Docker sandbox, and reports from the real git diff, merging only on approval.

The evaluation showed that the evidence-based approach works for what it is designed to check. Verification caught real build and data-model failures, reported environment failures honestly, and gave reviewers accurate reports. It also showed the boundary of that approach: a feature that passed every check still differed from its specification in ways a person could see at once, partly because of weaknesses inherited from earlier stages, and a manual fix was merged without being verified again. Verification and human review answer different questions, and the coding stage needs both.

9.2 Achievement of objectives

Table 9.1: Achievement of the objectives

Objective	Outcome	Status
O1: Validated code plan from approved artefacts	Planner with coverage validator and up to four targeted re-plans; maps_to claims not checked against code	Largely achieved
O2: Implementation on an isolated branch, with a path for models without tools	Feature branches, scoped tools, agentic loop, batch path with automatic capability detection	Achieved
O3: Sandboxed verification with feedback	Ten hard gates and five informational steps in Docker; up to three attempts with failure feedback	Achieved
O4: Reports from git, merge on approval, evaluation	Diff-based artefacts, merge, discard and undo; 410 tests, case study and nine stored versions	Achieved at limited scale

9.3 Individual research and product contributions

The main contribution is a tested design for the coding stage of a human-gated agent pipeline, in which the agent's work is driven by approved design artefacts, isolated in git, verified by real commands in a sandbox, and reported from the real diff. A second contribution is a set of deterministic checks, each grounded in a real failure of

generated code, that go beyond "does it build": unsatisfiable data models, unlinked pages, unguarded database calls and hardcoded connection strings. A third is an honest account, from stored runs, of what such verification cannot see, which is directly relevant to anyone who plans to rely on it.

9.4 Future directions

The most direct improvements follow from Chapter 6. The verifier should run again automatically whenever the feature branch changes after the last verification, so that the merge report always describes the code that will be merged. The plan validator should check that a requirement listed against a file has some trace in that file after coding, in the same spirit as the route check. And verification time should be reduced by caching installed packages between attempts.

Beyond these, the agent could check data types against the SRS, for example that a field the SRS calls a number is not stored as text, and could ask the QA Agent for acceptance tests derived from the SRS to run inside the loop, as AgentCoder [9] does with generated tests. When the architecture plan and the page designs disagree, as they did on routes, the agent should report the conflict instead of silently following one. Finally, the evaluation should be repeated with more features, more models and developers from outside the team.

REFERENCES

[1] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, *et al.*, "ChatDev: Communicative agents for software development," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers)*, 2024, pp. 15174–15186.

[2] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, S. Yau, Z. Lin, L. Zhou, *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024, pp. 23247–23275.

[3] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 50528–50652.

[4] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, *et al.*, "OpenHands: An open platform for AI software developers as generalist agents," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2025, pp. 65882–65919.

[5] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, "Executable code actions elicit better LLM agents," *arXiv preprint* arXiv:2402.01030, 2024.

[6] H. N. Phan, T. N. Nguyen, P. X. Nguyen, and N. D. Bui, "HyperAgent: Generalist software engineering agents to solve coding tasks at scale," *arXiv preprint* arXiv:2409.16299, 2024.

[7] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," *arXiv preprint* arXiv:2407.01489, 2024.

[8] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024, pp. 54107–54157.

[9] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, "AgentCoder: Multi-agent-based code generation with iterative testing and optimisation," *arXiv preprint* arXiv:2312.13010, 2023.

[10] W. Takerngsaksiri, J. Pasuksmit, P. Thongtanunam, C. Tantithamthavorn, R. Zhang, F. Jiang, J. Li, E. Cook, K. Chen, and M. Wu, "Human-in-the-loop software development agents," in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng.: Softw. Eng. Pract. (ICSE-SEIP)*, 2025, pp. 342–352.

[11] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language models: Survey, landscape, and vision," *IEEE Trans. Softw. Eng.*, vol. 50, no. 4, pp. 911–936, 2024.

[12] X. Zhu, W. Zhou, Q.-L. Han, W. Ma, S. Wen, and Y. Xiang, "When software security meets large language models: A survey," *IEEE/CAA J. Autom. Sinica*, vol. 12, no. 2, pp. 317–334, 2025.

[13] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, pp. 1–79, 2024.

[14] J. He, C. Treude, and D. Lo, "LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, pp. 1–30, 2025.

[15] K. Pelluru, "LangChain & LangGraph in production: Architectures for multi-agent LLM systems," *Journal of Data and Digital Innovation (JDDI)*, vol. 2, no. 3, pp. 1–9, 2025.

[16] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[17] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen, "RepoCoder: Repository-level code completion through iterative retrieval and generation," in *Proc. 2023 Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2023, pp. 2471–2484.

[18] Stack Overflow, "2025 Developer Survey: AI," 2025. [Online]. Available: https://survey.stackoverflow.co/2025/ai (accessed Oct. 8, 2026).

[19] "SLASSCOM reflects on a year of creating opportunities, strengthening Sri Lanka's knowledge and innovation industry," Daily FT, Jul. 15, 2026. [Online]. Available: https://www.ft.lk/it-telecom-tech/SLASSCOM-reflects-on-a-year-of-creating-opportunities-strengthening-Sri-Lanka-s-knowledge-and-innovation-industry/50-794672 (accessed Oct. 8, 2026).

[20] "Data protection compliance regime takes effect on 1 Jan. 2027," Daily FT, 2026. [Online]. Available: https://www.ft.lk/front-page/Data-protection-compliance-regime-takes-effect-on-1-Jan-2027/44-795776 (accessed Oct. 8, 2026).

[21] GitHub, "GitHub Copilot plans and pricing." [Online]. Available: https://github.com/features/copilot/plans (accessed Oct. 8, 2026).

[22] Anysphere, "Cursor pricing." [Online]. Available: https://cursor.com/pricing (accessed Oct. 8, 2026).

[23] Lovable, "Subscription plans." [Online]. Available: https://docs.lovable.dev/introduction/subscription-plans (accessed Oct. 8, 2026).

[24] StackBlitz, "Bolt pricing." [Online]. Available: https://bolt.new/pricing (accessed Oct. 8, 2026).

[25] Exchange-Rates.org, “USD to LKR: Convert US Dollars to Sri Lanka Rupees.” [Online]. Available: https://www.exchange-rates.org/converter/usd-lkr (accessed Oct. 7, 2026).

GLOSSARY

Batch path	The coding path that makes one model call per planned file, for models that cannot use tools.
Branch	An isolated line of development in git; each feature has its own.
Code plan	The list of files to create, modify or delete, with the requirements each serves.
Coding loop	The tool-using agent that implements the code plan in the workspace.
Diff	The exact set of changes between the feature branch and main.
Gate (hard)	A verification step whose failure marks the whole attempt as failed.
Informational step	A verification step reported to the reviewer that never fails an attempt on its own.
Live preview	The built feature running in a container so the reviewer can try it.
Merge report	The report listing verification results, changed files and the diff for one version.
Mongoose schema	The definition of a MongoDB document's fields, types and required fields.
No-fast-forward merge	A git merge that always records a merge commit, keeping the feature identifiable.
Project manifest	The record of routes, endpoints and models built so far in a project.
Route Handler	A Next.js file that implements API methods for one URL path.
Sandbox	A disposable Docker container in which generated code is installed, built and run.
Scaffold	The working starter project created for every new project repository.
Workspace	A project's git repository on disk, which all tools are confined to.

APPENDICES

Appendix A Research instruments or extended results

A.1 Test catalogue

Table A.1 : Test files for the component

Test file	Functions	What it covers
test_coder_agent_retries	14	Attempts, step limit, crashes, final failure
test_coder_agent_revise, _stream	34	Revision flow; streamed events
test_coder_agent_batch_generation	8	Batch path
test_coder_agent_keyword_matching, _well_specified_files	31	Finding target files for revisions
test_coder_coding_loop_task_message, test_coder_prompt	39	Task and prompt construction
test_coder_planner_exploration	6	Exploration planner for revisions
test_coder_env_uri, test_revision_file_tokens	26	Connection strings; file names in comments
test_model_capabilities	9	Tool-call capability detection
test_coder_plan_validator, _diff_builder	19	Plan coverage; diff-based artefacts
test_coder_verify, _verify_mongodb_gates	23	Verification steps and database gates
test_route_checker, test_nav_checker, test_render_checker	28	Routes, reachability, page rendering
test_coder_schema_form_checker, _hardcoded_secret_checker	17	Schema/form coverage; hardcoded URIs
test_coder_db_fallback_checker, _functional_checker	21	Database fallback; CRUD smoke test
test_coder_relevance_scan, _ui_expectations_checker, _step	18	Informational word-overlap checks
test_style_checker	7	Styling scan
test_coder_tools	35	Tool behaviour and containment
test_workspace_scaffold, _nextjs_tailwind, _preview_route_announcer	37	Scaffold and upgrades
test_workspace_env_local, _touched_files_since, _undo_merge	18	Environment file, touched files, undo
test_preview_service	16	Live preview
test_feature_code_with_qa_report_download	4	Code export with QA report
Total	**410**	

A.2 Verification summary of the retail store feature, version 1

The lines below are taken from the merge report the agent saved, with long outputs shortened.

```
Verification: PASSED
Coding attempts used: 3
- next.config.mjs integrity: passed
- npm install: passed
- next build: passed
- server boot (next start + /api/health): passed
- npm run test (root): skipped
- endpoint route coverage: passed
- database null-guard coverage: passed
- hardcoded MongoDB URI scan: passed
- schema/form field coverage: passed
- page reachability: passed
- home page render: passed
- feature page render: info (1 page, HTTP 200)
- CRUD functional smoke test: info (POST returned 400)
```

Appendix B Academic integrity and AI use disclosure

Table B.1 : Use of generative AI tools

Activity	**Tool**	**How it was used**	**How the output was checked**
System runtime	qwen3-coder via Ollama	The model inside the Coder Agent, as described in this thesis	Sandboxed verification, tests and human review
Figures	Claude (Anthropic)	Drawing the design diagrams from descriptions of the implementation	Checked against the implementation by the author